

Computer Science Fundamentals

Introduction

Technische Hochschule Rosenheim
Winter 2025/26
Prof. Dr. Jochen Schmidt

- Lecture: 4 SWS (= 2 x 90min per week)
- Tutorial: 2 SWS, split into groups
 - in English (for AAI students): J. Schmidt
 - in German (for INF students): D. Stecher
 - register via Learning Campus
- Workload: 210h (90h lecture/exercises, 120h additionally for studying on your own)
- Slides, exercises etc.: Download via Learning Campus
- Exam
 - written exam at the end of the semester
 - 90 minutes
 - permitted aids: non-programmable calculator

Chapter 1: *Introduction to Computer Science*

Hardware Basics

Chapter 2: *Number Representation*

2.1 Number Systems & Conversion

2.2 Addition & Subtraction, Negative Integers

2.3 Multiplication/Division, Floating Point Numbers

Chapter 3: *Information and Source Coding*

3.1 Information: Basics on entropy,
mean word length, redundancy

3.2 Huffman/Fano Algorithms

3.3 Arithmetic Coding

3.4 Run-Length Encoding
LZW Compression

Chapter 4: *Code Protection and Channel Coding*

4.1 Fault-tolerant Codes introduction,
Hamming Distance, m-out-of-n codes, parity

4.2 Hamming Codes

4.3 Cyclic Redundancy Check (CRC)

4.4 Reed-Solomon Codes

Chapter 5: *Cryptography*

5.1 Basics, classic methods

5.2 Modern methods
AES, Diffie-Hellman, RSA, Elliptic Curves

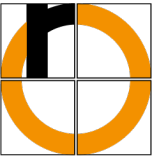
Chapter 6: *Graph Theory*

6.1 Basics, directed and undirected graphs

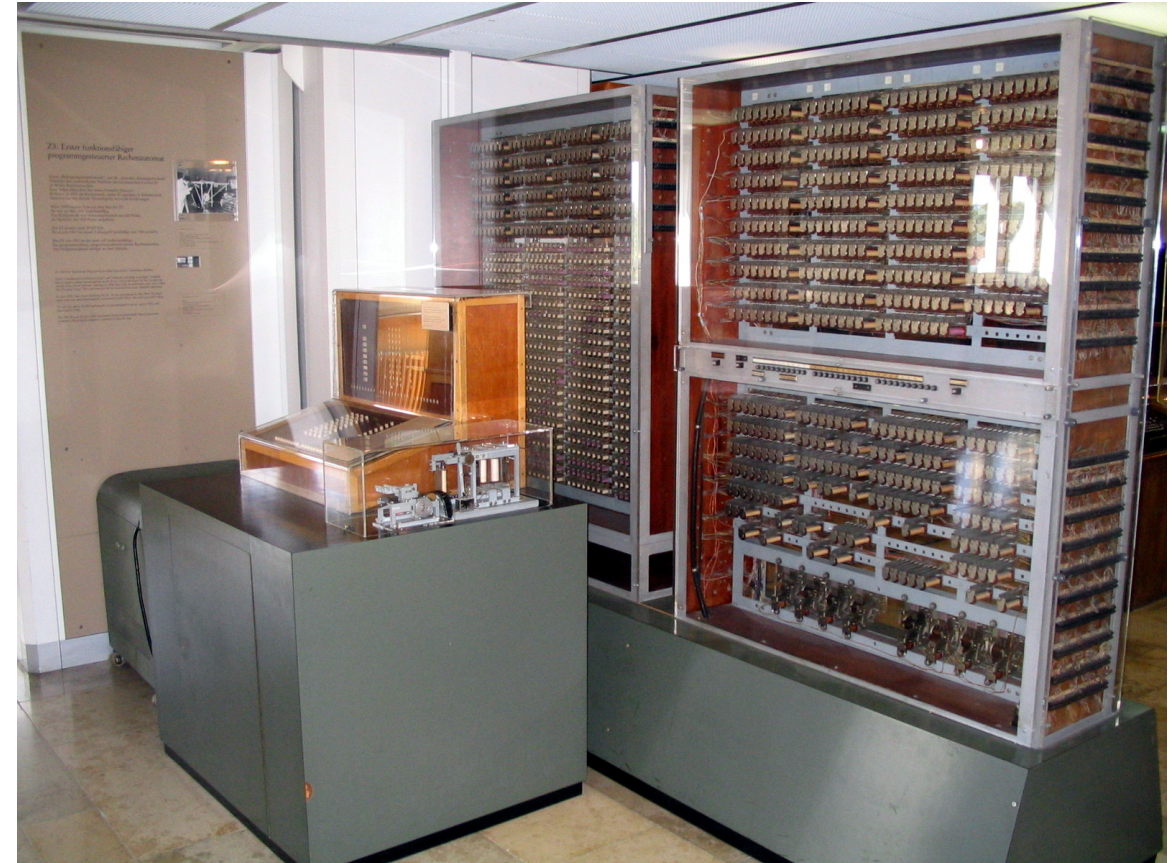
6.2 Trees, graph-search, A*-algorithm

- in German
 - H. Ernst, J. Schmidt, G. Beneken. **Grundkurs Informatik**. Springer Vieweg, 8. Aufl. 2023.
 - J. Schmidt. **Grundkurs Informatik – Das Übungsbuch: 163 Aufgaben mit Lösungen**. Springer Vieweg, 3. Auflage 2023.
 - H. Herold, B. Lurz, J. Wohlrab, M. Hopf. **Grundlagen der Informatik**. Pearson Studium, 4. Aufl. 2023.
 - D.W. Hoffmann. **Einführung in die Informations- und Codierungstheorie**. Springer Vieweg, 2. Aufl., 2024.
 - C. Paar, J. Pelzl. **Kryptografie verständlich: Ein Lehrbuch für Studierende und Anwender**. Springer Vieweg, 2016.
- in English
 - J.G. Brookshear, D. Brylow. **Computer Science: An Overview**. Pearson, 13th edition, 2020.
 - A. Neubauer, J. Freudenberger, V. Kühn. **Coding Theory: Algorithms, Architectures and Applications**. Wiley, 2011.
 - S. Rubinstein-Salzedo. **Cryptography**. Springer Undergraduate Mathematics Series. Springer, 2018.
 - C. Paar, J. Pelzl. **Understanding Cryptography: A Textbook for Students and Practitioners**. Springer, 2014.

The First Computer



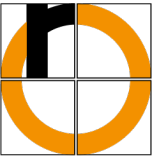
- Built in the 1940s by Konrad Zuse in Berlin
- Electromechanical Z3:
approx. 2600 Relays and
64 memory cells, 22 Bits each
- Multiplication takes about 3 seconds



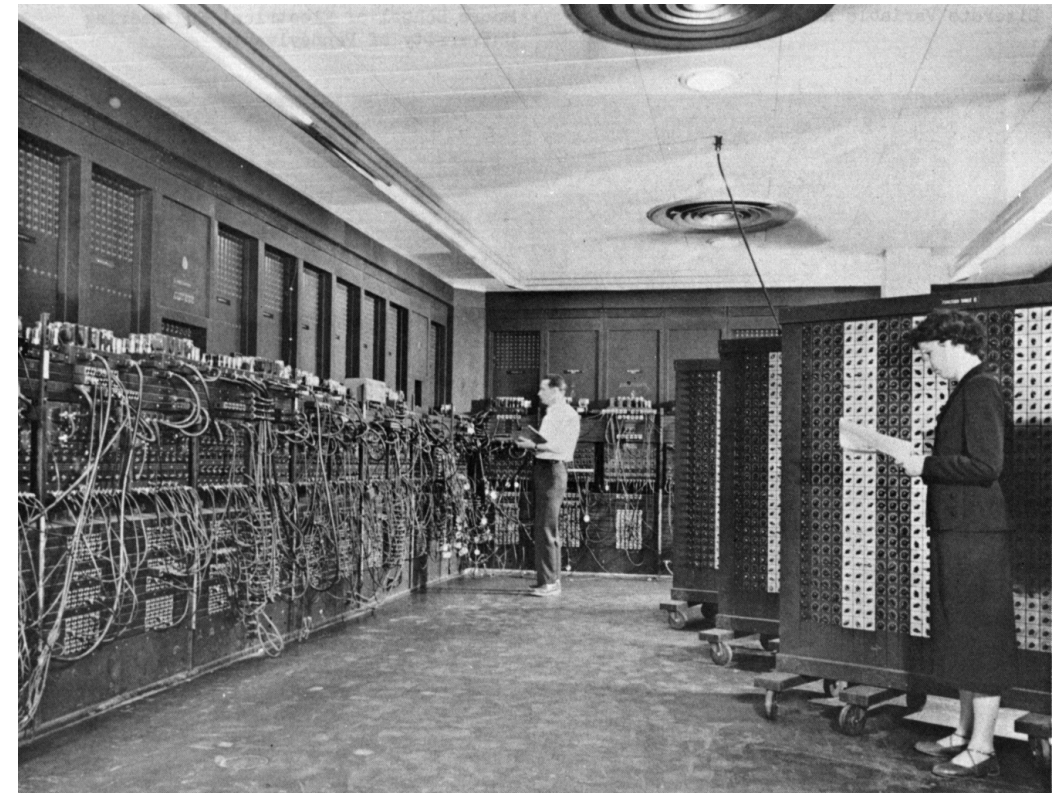
Z3 replica in the Deutsches Museum, Munich

© [Venusianer](#), [Z3 Deutsches Museum](#), [CC BY-SA 3.0](#)

First Electronic Vacuum Tube Computer



- The first electronic computer ENIAC
(Electronic Numerical Integrator and Automatic Calculator)
1946 in the USA by J.P. Eckert and J.W. Mauchley
 - approx. 17,000 vacuum tubes and 1500 relays
 - weight: 30 tons,
floor space required: 140 m²,
power requirement: 140 kW.

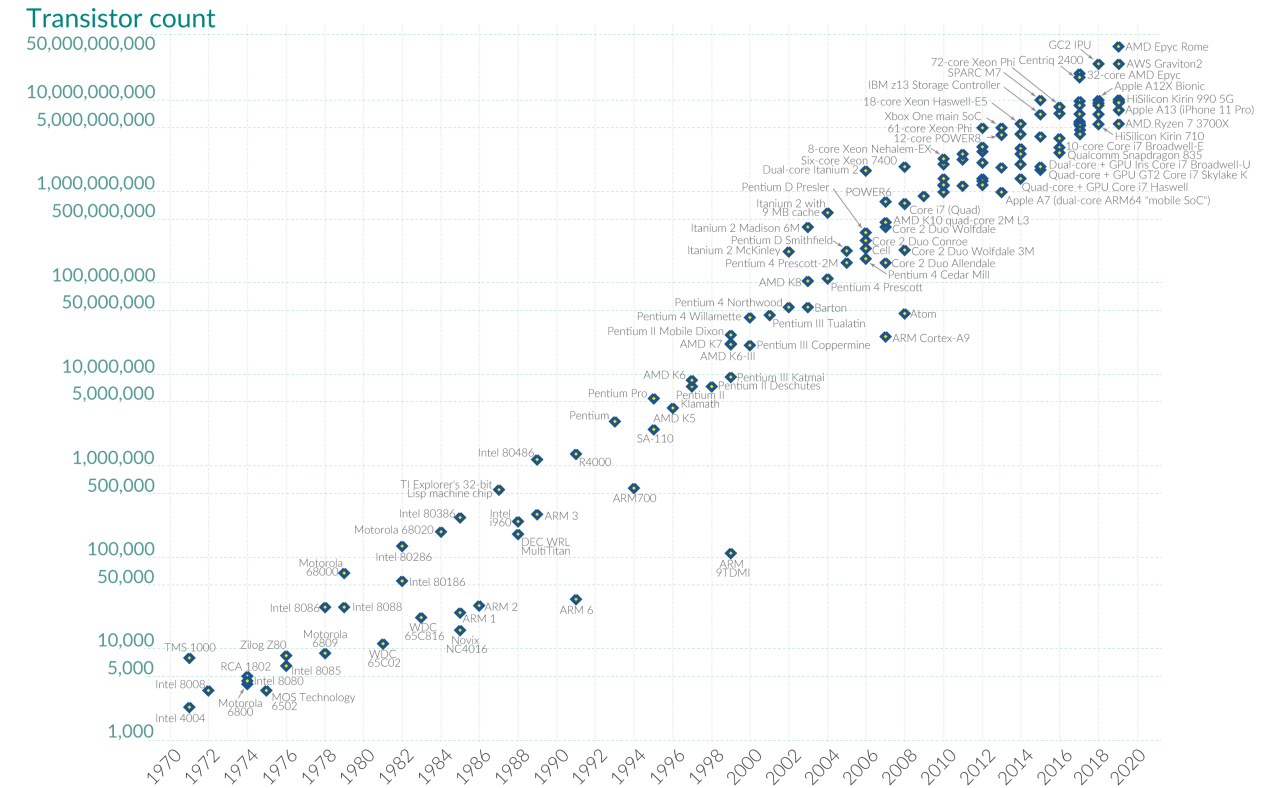


- Modern Computer
 - Microprocessors with millions of transistors
 - Main memory with billions of memory cells (Gigabytes)
 - Billions of operations per second
- Moore's Law says that
 - the packing density of transistors on a microprocessor
 - doubles about every 18 months.

Moore's Law: The number of transistors on microchips doubles every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Our World
in Data

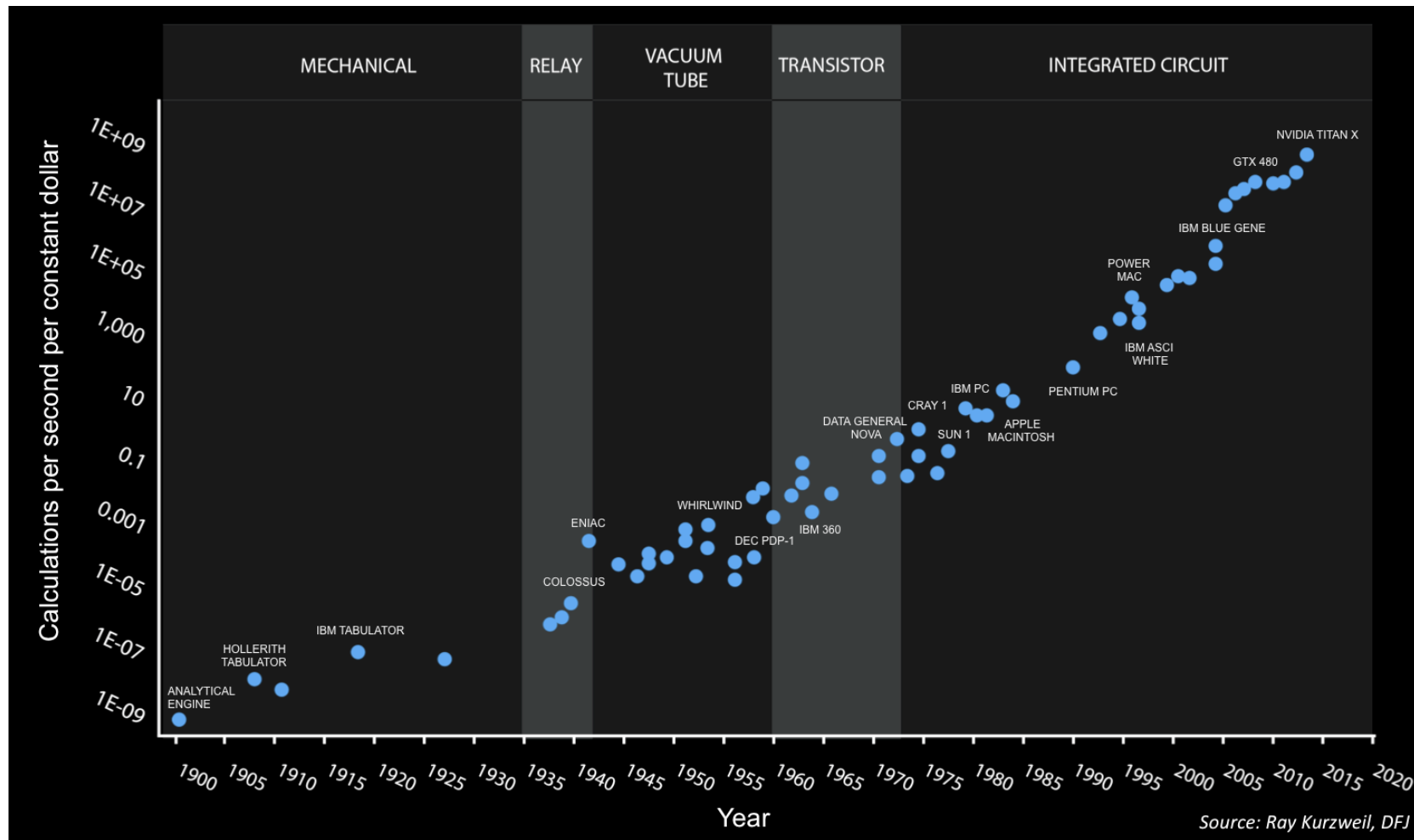
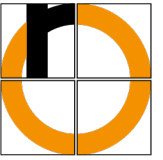


Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

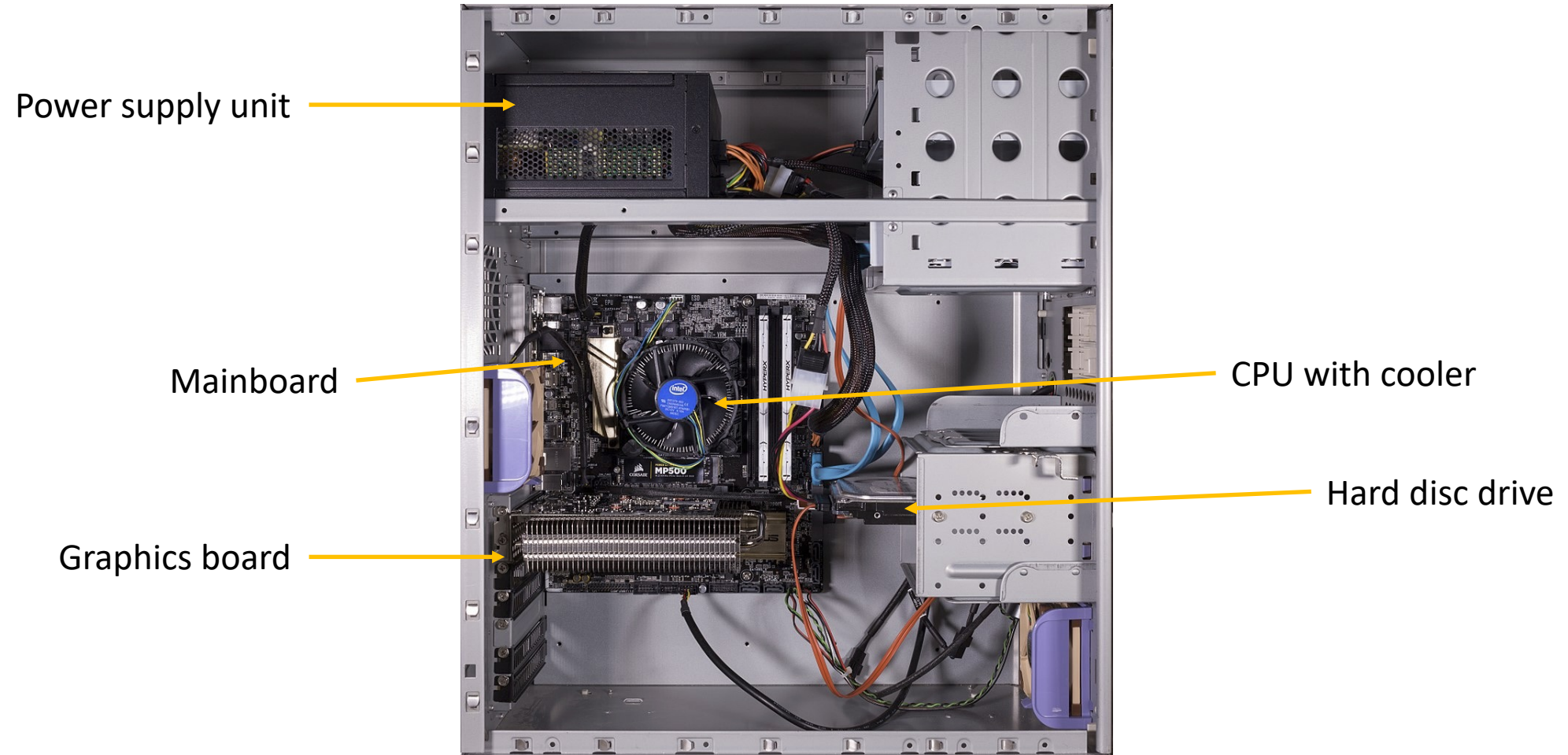
OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

Moore's Law cont.



© Steve Jurvetson, [Moore's Law over 120 Years](#), CC BY 2.0



© Tobias "ToMar" Maier, [2017 mid range pc in late 1990s case](#), CC BY-SA 3.0

19" Server Rack



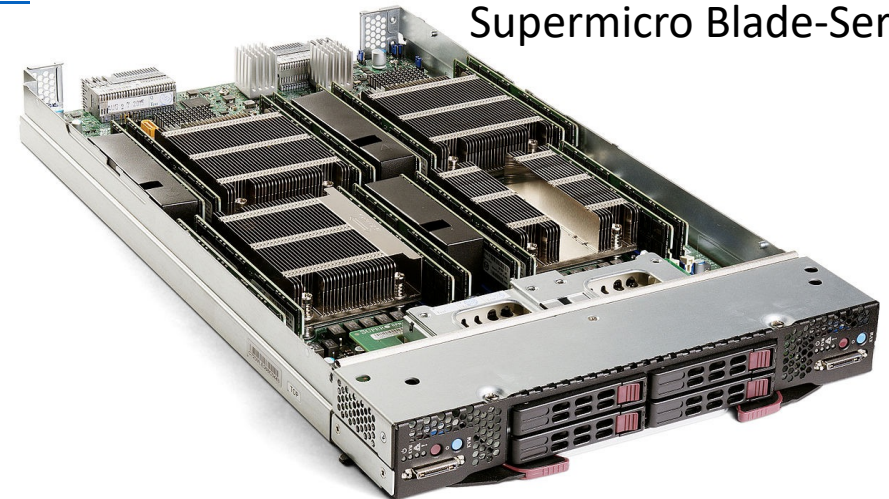
© [Jfreyre, Rack001](#), [CC BY-SA 3.0](#)

Dell Webserver



© [Rodzilla](#) at [English Wikipedia](#), [Inside and Rear of Webserver](#), [CC BY 2.5](#)

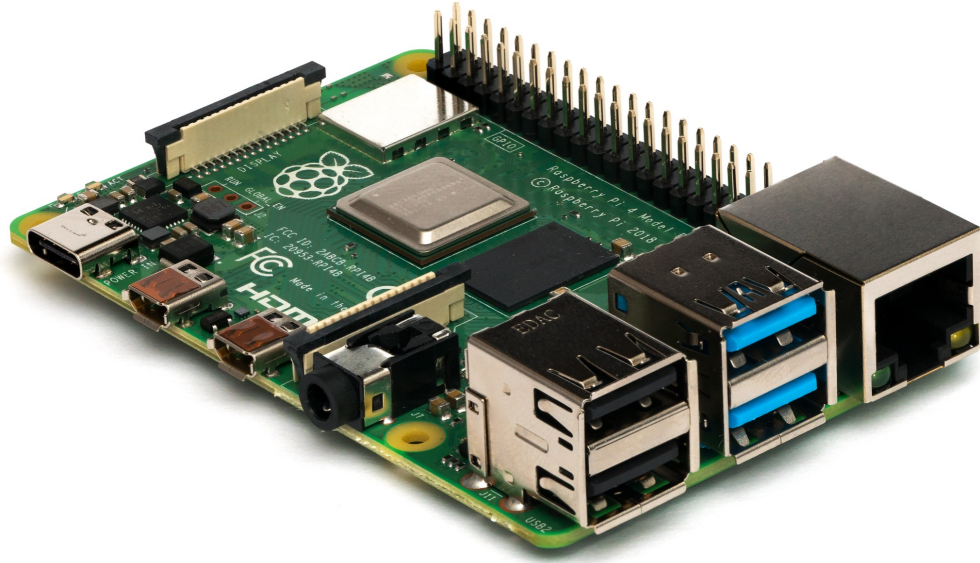
Supermicro Blade-Server



© [Dmitry Nosachev](#), [Supermicro SBI-7228R-T2X blade server](#), [CC BY-SA 4.0](#)

Single-board Computer

Raspberry Pi 4 B



© Miiichiaieil Hieinizilieir / [Wikimedia Commons](#) / [CC BY-SA 4.0](#), [Raspberry Pi 4 Model B - Side](#), [CC BY-SA 4.0](#)

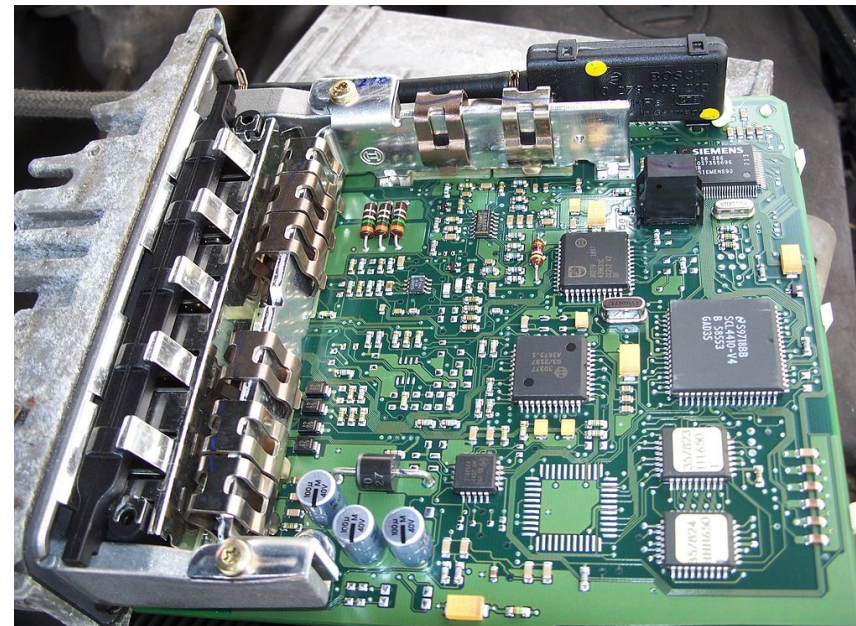
Arduino Uno R3



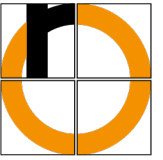
© [SparkFun Electronics](#) from Boulder, USA, [Arduino Uno - R3](#), [CC BY 2.0](#)

- A much larger market are embedded systems (*eingebettete Systeme*)
- Application areas include:
 - aircraft
 - cars
 - trains
 - TVs
 - Blu-Ray players
 - washing machines
 - hearing aids
 - computer tomographs
 - ...

Engine Control Unit VW Golf TDI



© [Cschirp](#), [Motorsteuerung VW Golf TDI innen](#), [CC BY-SA 3.0 DE](#)



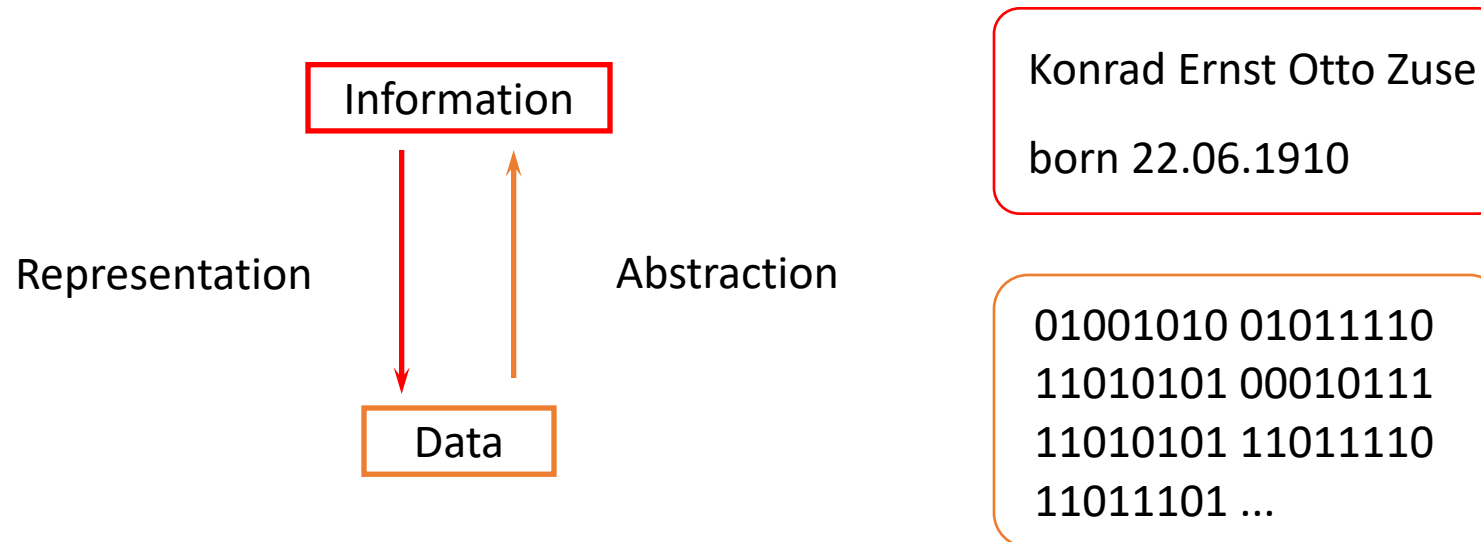
- Car = computer network on wheels
 - upper class: up to 80 control units (*Steuergeräte*)
 - midsize: > 20 control units
 - connected via a bus system (CAN/FlexRay)
- Tasks, e.g.,
 - Engine control
 - Instrument cluster
 - ABS/ESP
 - Airbag
 - Driver assistance systems



Airbag control unit

- and these computers obviously require software to work ...

- For the purpose of (formal) processing (e.g., using a computer)
 - information is represented by **data**
 - and stored as such



- Data are represented by zeros and ones in the computer

Bit (*Binary Digit*)

- Alphabet = {0, 1}
 - it can be proven that this is sufficient to compute anything that is computable (→ Theoretical Computer Science, summer semester)
- Required, as this is technologically simple, e.g., using
 - electric charge (0 = uncharged; 1 = charged)
 - voltage (0 = 0 Volt; 1 = 5 Volt)
 - magnetization (0/1 depending on the polarity of the magnetization)

- 2-bit sequence
 - 4 possibilities (00, 01, 10, 11)
- 3-bit sequence
 - 8 possibilities (000, 001, 010, 011, 100, 101, 110, 111)
- In general: Each additional bit **doubles** the number of possible bit sequences.
→ There are exactly 2^N possible bit sequences of length N .

- Computers only work with groups of bits
 - typical: 8 Bits, 16 Bits, 32 Bits or 64 Bits
- Byte
 - technically: the smallest amount of data that can be addressed via the address bus
 - usually: a **group of 8 bits** is called a byte
- A single byte can be used to store, e.g.:
 - a positive integer between 0 and 255,
 - an integer between -128 and $+127$,
 - a character (using a character code like ASCII)
 - the brightness of a point in an image called “pixel” (= picture element)

- A **file** (*Datei*) is an arbitrarily long sequence of bytes (stored on disk)
- Size of a file = number of **bytes** it contains
Units of measurement:

k	=	1024	=	2¹⁰	≈	10³	(k = Kilo)
M	=	1024²	=	2²⁰	≈	10⁶	(M = Mega)
G	=	1024³	=	2³⁰	≈	10⁹	(G = Giga)
T	=	1024⁴	=	2⁴⁰	≈	10¹²	(T = Tera)
P	=	1024⁵	=	2⁵⁰	≈	10¹⁵	(P = Peta)
E	=	1024⁶	=	2⁶⁰	≈	10¹⁸	(E = Exa)

- Nomenclature inconsistent, sometimes the familiar metric values are also used (e.g., for hard drives)

- For clearer differentiation: the use of other prefixes has been proposed in 1996
- Defined in standard (*Norm*) IEC 80000-13:2008
- Still not fully established yet (but used more and more often)

Kibibyte (**KiB**) = 2^{10} Byte (Ki = Kilo, bi = binary)

Mebibyte (**MiB**) = 2^{20} Byte (Me = Mega)

Gibibyte (**GiB**) = 2^{30} Byte (Gi = Giga)

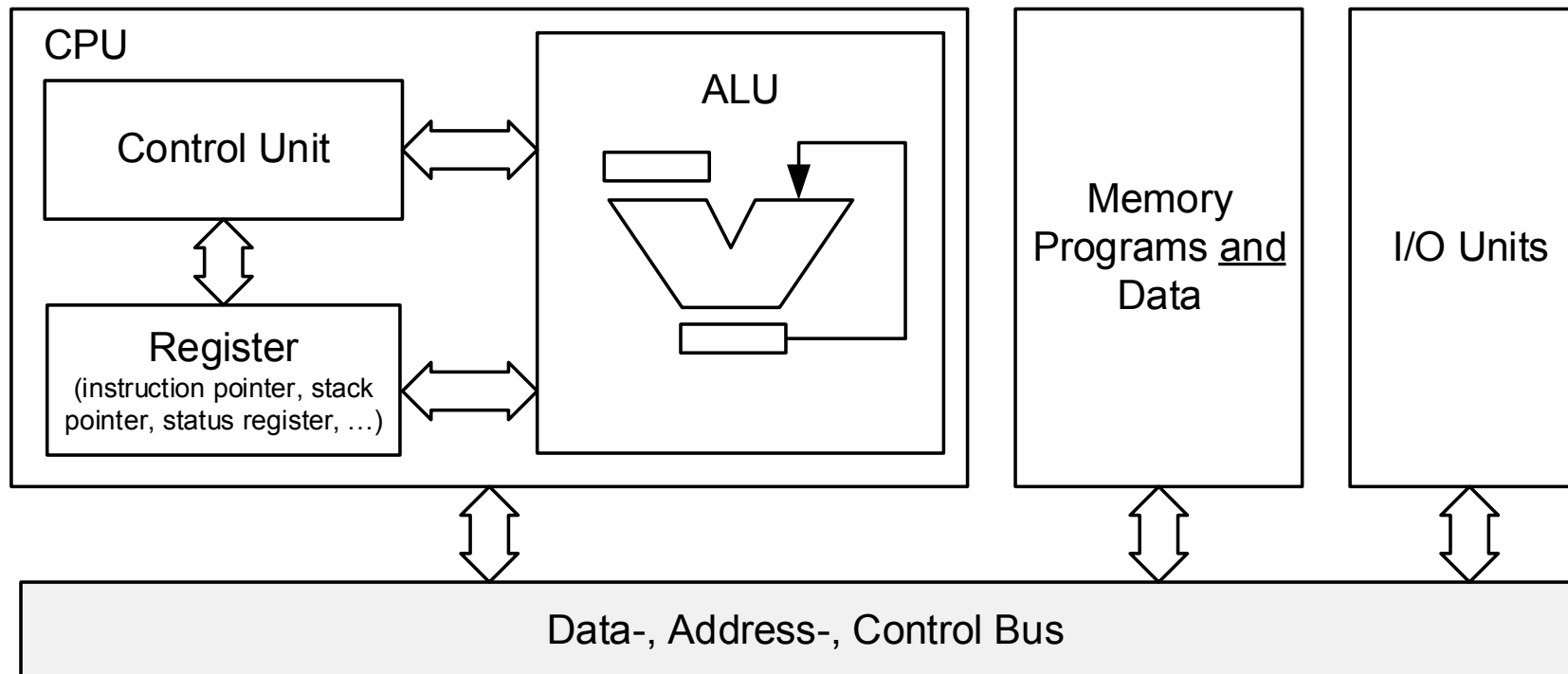
Tebibyte (**TiB**) = 2^{40} Byte (Te = Tera)

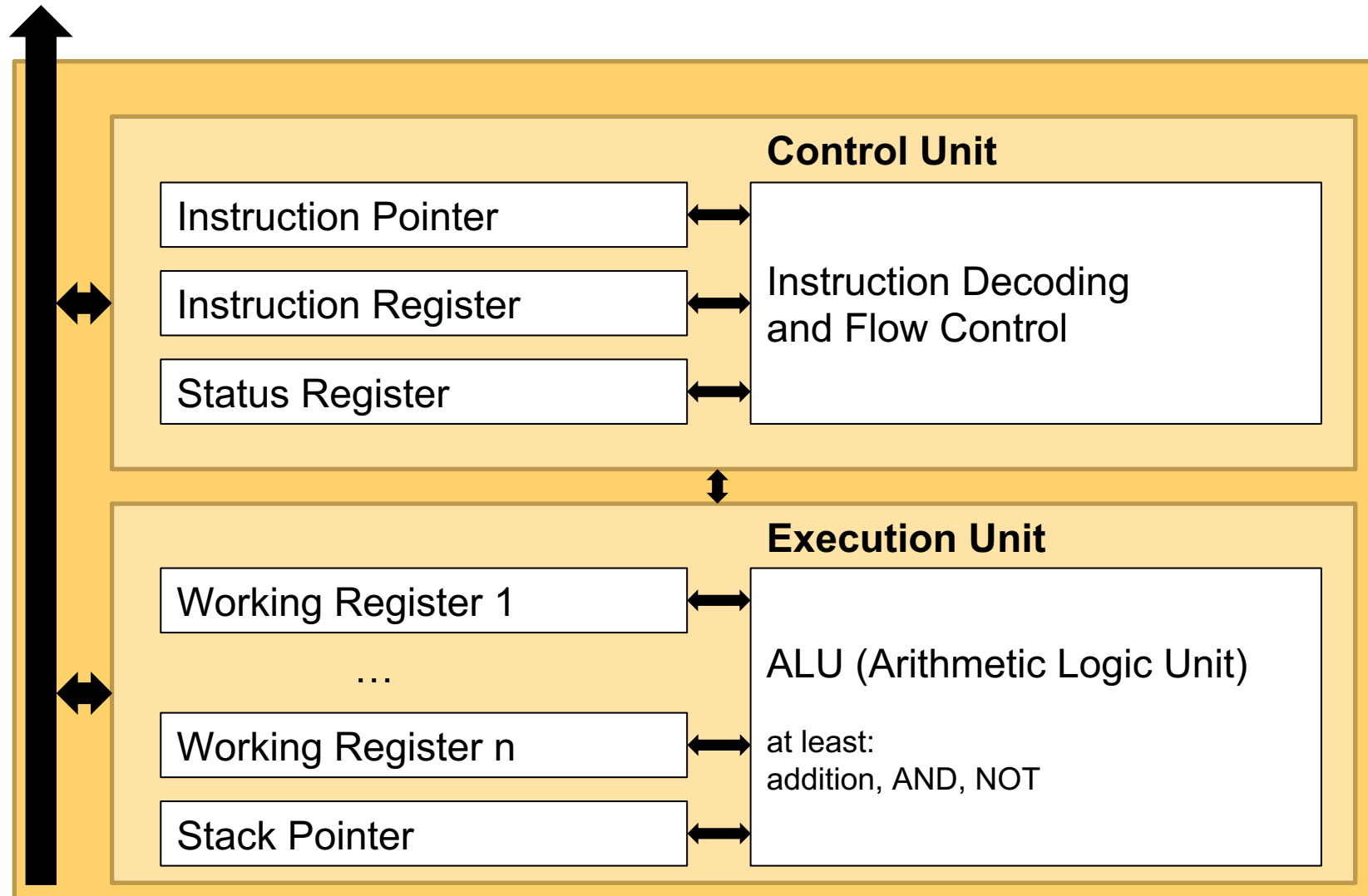
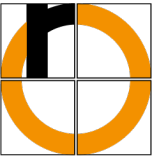
- Processor
 - CPU = „Central Processing Unit“
 - “brain” of a computer
 - Tasks: Execution of programs, control and management of Hardware
- RAM (Random Access Memory)
 - main memory (*Hauptspeicher*)
 - contains programs to be executed
 - as well as the data
 - volatile memory (*flüchtiger Speicher*)
- ROM (Read Only Memory)
 - contains BIOS (Basic Input Output System), which
 - initializes the system,
 - tests the most important hardware components,
 - and boots the operating system (*Betriebssystem*) from hard drive/SSD
 - non-volatile (*nichtflüchtig*)
- Busses and interfaces
 - communication between mainboard components
 - connection of peripheral devices (graphics boards, network, storage devices, printer, etc.)
- Chipset
 - Circuitry for controlling the mainboard components

- **Processor (CPU)** consisting of
 - **Control Unit** (*Steuerwerk*) reads commands and operands from memory and interprets these using an instruction table
 - **Execution Unit** (*Rechenwerk*) executes arithmetic, floating point, and logic operations
- **Main Memory** contains program commands **as well as** data
- **Input-/Output (I/O) Units** input programs and data, output processed data
- **Bus System** used for transportation of data between CPU, RAM, and I/O units

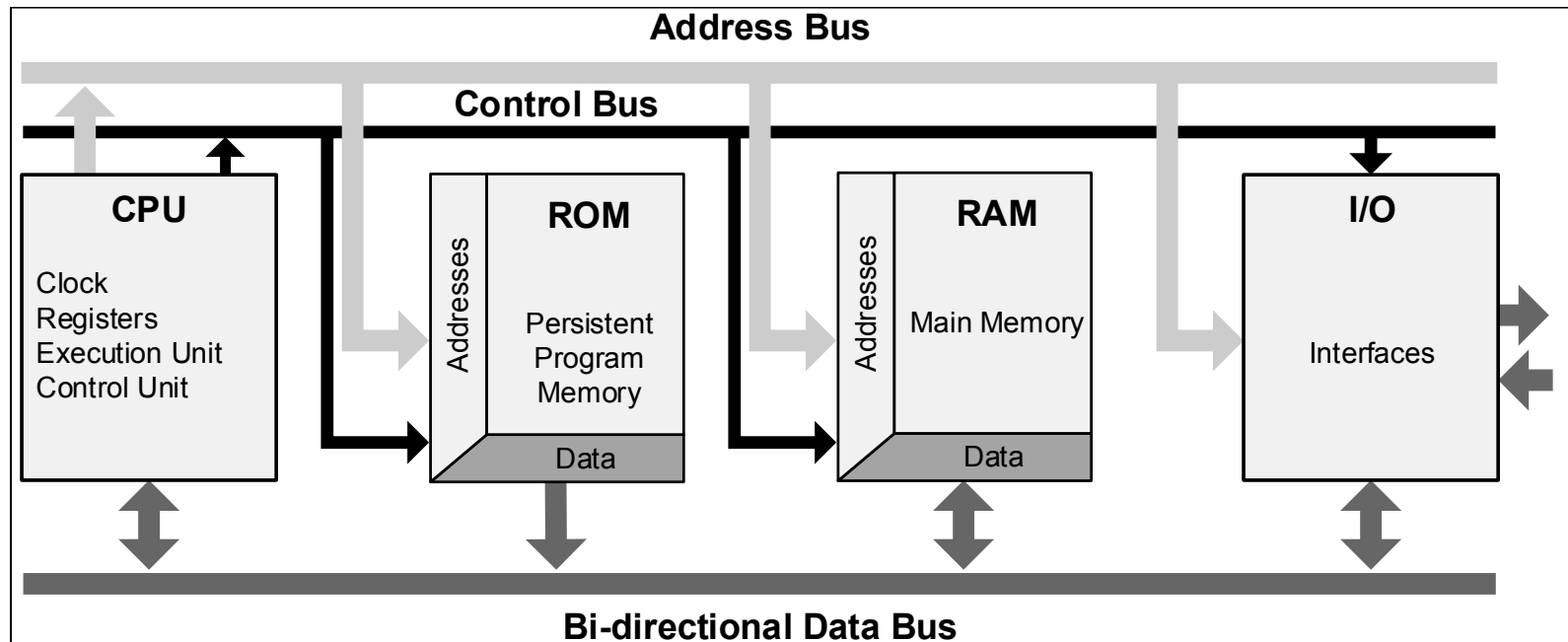
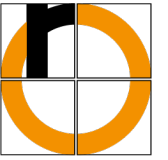
John von Neumann developed the fundamental principles of a computer system in the mid 1940s:

- Control unit, execution unit, I/O, connections (busses)
- Program and data stored in memory
- Step by step execution of commands
- Conditional jumps (*Sprünge*) and branching (*Verzweigungen*)





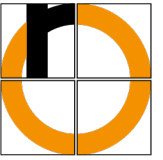
- Registers are CPU-internal memory cells
 - any command, data or address operand must pass through a register in order to be processed
 - not to be confused with the Cache!
- Properties and width (= size in bits) vary for different CPUs and have an impact on performance
- Types of registers
 - Working register (*Arbeitsregister*)
Contain data and addresses
 - Instruction Pointer (or: Program Counter; *Befehlszählerregister*)
Address of the next instruction scheduled for execution
 - Instruction register (*Befehlsregister*)
Contains the currently executed (binary) machine code instruction
 - Status register
Information regarding results of the executed instruction
 - Stack Pointer (*Stapelzeiger*)
Address of the topmost stack element



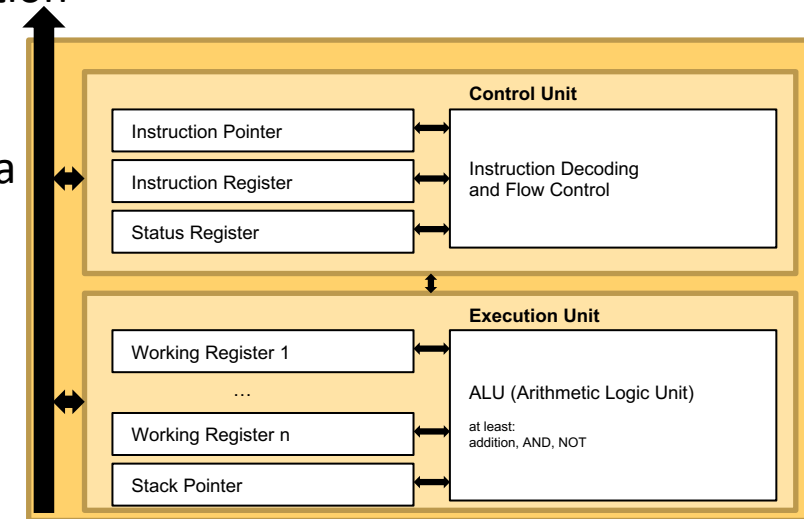
The CPU is connected to other components via **Busses**

- **Data Bus** (*Datenbus*) for data exchange with main memory (bi-directional)
- **Address Bus** (*Adressbus*) for transferring the proper memory address (uni-directional)
- **Control Bus** (*Steuerbus*) for controlling the data-/address busses

Processor – Instruction Cycle



1. The instruction pointer contains the address of the next machine code instruction
→ **Address of instruction is transferred** to main memory via the address bus.
2. The **instruction is transferred** from main memory to the instruction register via the data bus.
3. The **decoder analyzes** the instruction and initiates its execution.
4. The **instruction is executed**
→ depending on the instruction, additional steps may be required, like: reading additional data from main memory, driving peripheral interfaces, computations in the ALU, ...
5. In case the instruction was a **jump/branching** operation, the **instruction pointer is set** to the appropriate new address; **otherwise**, the **instruction pointer is incremented** to point to the following instruction (i.e., by 1 or the appropriate amount, depending on the length of the instruction that was executed).
6. The CPU proceeds with step 1.



Word width (= how many bits are available in registers and busses; *Wortbreite*) of

- Working/Data registers
 - maximum size of integers and size/accuracy of floating-point numbers (*Gleitkommazahlen*)
- Data bus
 - number of bits that can be read or written in parallel from or to main memory
- Address bus
 - maximum size of a memory address, and therefore the maximum size of RAM

Clock rate (*Taktfrequenz*)

- everything in a digital system is done according to a central clock
- can be used to judge the speed of a CPU
 - note: ranking by clock rate is only valid for CPUs having the same architecture
- different components may have different clock rates
 - these are derived from a basic clock rate of the mainboard
 - the internal CPU clock rate is typically much higher than that of other components
 - Example: 133 MHz basic clock rate of the mainboard
 - multiplier of 20 → 2.66 GHz CPU clock rate
 - this obviously can lead to issues like the CPU having to wait for other components (see: Cache)

- Decimal units are used

m	=	1/1000	=	10^{-3}	(m = milli)
μ	=	1/1000000	=	10^{-6}	(μ = mikro)
n	=	1/1000000000	=	10^{-9}	(n = nano)
p	=	...	=	10^{-12}	(p = pico)
f	=	...	=	10^{-15}	(f = femto)

- Example

- 2.6 GHz CPU has a clock rate of $2.6 \cdot 10^9 = 2,600,000,000$ Hertz (oscillations per second)
- a single clock pulse therefore lasts $1/(2.6 \cdot 10^9) \text{ s} = 0.38 \cdot 10^{-9} \text{ s} = 0.38 \text{ ns}$

Metrics

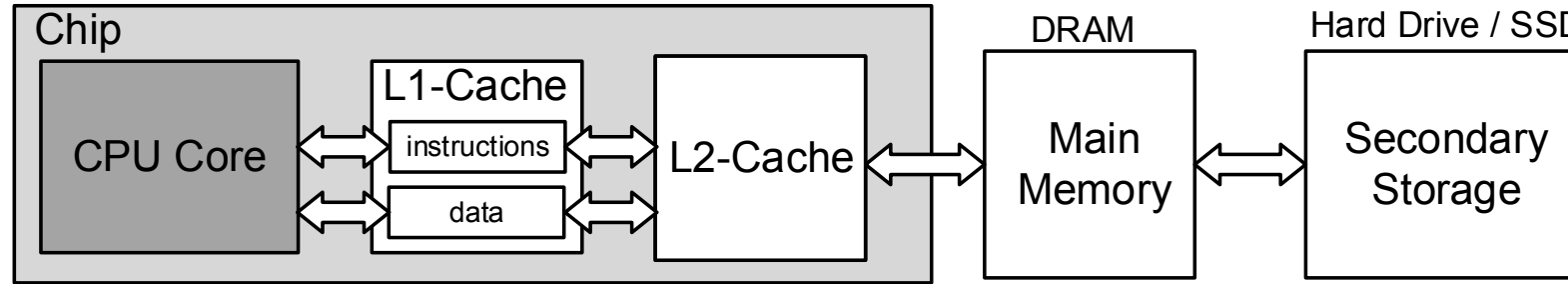
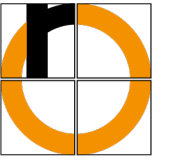
- MIPS (Million Instructions per Second)
 - number of instructions (in millions) that a CPU can execute per second
 - not considering floating-point operations
- FLOPS (Floating-Point Operations Per Second)
 - number of floating-point operations that a CPU can execute per second
- Other aspects that are concerned with peripheral devices or installed software
 - e.g., hard drive access times, transfer times (drives or network), costs, ...

- **DRAM** (Dynamic RAM)
- Typical area of application: Main memory
- Technology: Capacitor + Transistor
(0 = uncharged, 1 = charged)
- Low space consumption on the chip, inexpensive
- Slow
- Capacitor loses its charge over time → memory content needs to be refreshed regularly
(refresh → **dynamic**)

- **SRAM** (Static RAM)
- Typical area of application: registers, cache
- Technology: Flip-flops (up to 6 transistors)
- Large space consumption on the chip, expensive
- Fast (registers work at CPU clock rate)
- No refresh necessary (→ **static**)

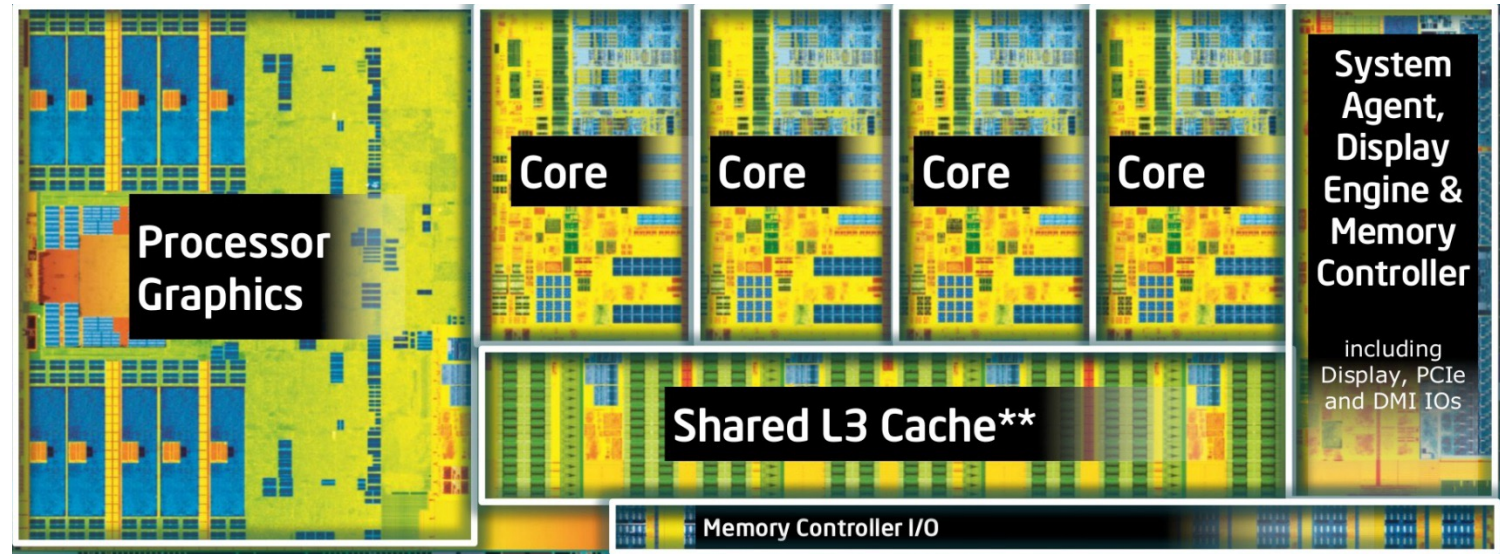
- Problem:
CPU is very fast – main memory very slow
- CPU would have to wait for main memory every time memory is accessed (commands/data)
- Observation
 - Programs and data are temporally and spatially **local**
 - **temporal:**
 - the same commands are executed in a loop over and over again
 - the same data is accessed over and over again
 - **spatial:**
 - related data and program parts are not wildly scattered in memory, but located one after the other

→ it is worthwhile to load larger blocks at a time from the main memory



- **Cache** = fast intermediate memory
- Typical: hierarchical (levels 1 to 3), getting bigger and bigger
- Level 1
 - Commands and data managed separately
 - Commands and data are read/written individually
- from Level 2:
 - Management of memory blocks
 - Typical block-size: 16, 32, 64 bytes

Example: Intel Core i7 4770k



© Intel

- Level 1
 - separately for each core
 - two times 32 KB (Data and instructions separately)
- Level 2
 - separately for each core
 - 256 KB
- Level 3
 - shared for all cores and GPU
 - 8 MB

- Problem:
Inconsistencies when multiple CPUs/cores access the same data in memory at the same time
- Example:
 - 2 cores read the same address of the main memory and store it in their cache
 - at this time, the content of this address exists three times:
The original in main memory, and two copies in the two caches
 - with multiple cache levels, the problem continues to exacerbate
 - If a core modifies the content:
 - the copy in its cache is changed,
 - the cache may write the change directly to the main memory (Write-through cache)
 - the copy in the cache of the second core now contains a deprecated value

→ Hardware must synchronize caches

- This chapter provides a coarse overview
- Computer architecture will be discussed
 - at a deeper level in the “IT-Systeme” and “Rechnerarchitektur” classes (2nd and 4th semester INF)
 - somewhat deeper in “IT-Systems” (1st semester AAI)